P3 - Genetic Algorithm Integer Representation

Sistemas Inteligentes

Profesores
Javier Jareño Dorado
Elisa Guerrero Vázquez

16 de marzo de 2026

Universidad de Cádiz
Grado en Ingeniería Informática - Especialidad Computación

1. Problem Description: JSSP

The Job Shop Scheduling Problem (JSSP) is a classical combinatorial optimization problem in which J jobs must be processed on M machines. Each job j consists of an ordered sequence of operations, where each operation must be executed on a specific machine for a given processing time. The goal is to determine a schedule that minimizes the total completion time of all jobs, known as the *makespan* ($C_{\text{máx}}$).

1.1. Instance Definition

An instance is fully specified by two matrices of size $J \times M$:

- **Machine routing matrix** $\mathbf{R} \in \{0, 1, \ldots, M\}^{J \times M}$: entry $R_{j,k}$ indicates the machine assigned to the k-th operation of job j. A value of 0 means that job j has fewer than k operations (i.e. the entry is unused).
- **Processing time matrix** $\mathbf{T} \in \mathbb{N}_0^{J \times M}$: entry $T_{j,k}$ is the processing time of the k-th operation of job j. If $R_{j,k} = 0$, then $T_{j,k} = 0$.

Let n_j denote the number of operations of job j, defined as:

$$n_j = \text{máx}\{k \mid R_{j,k} \neq 0\}, \quad j = 1, \ldots, J \tag{1}$$

The total number of operations across all jobs is:

$$N_{op} = \sum_{j=1}^{J} n_j \tag{2}$$

Listing 1: Example instance with 3 jobs and 3 machines

```
% Machine routing matrix (which machine for each operation)
M_maquinas = [
    1, 2, 3;   % Job 1: M1 -> M2 -> M3       (3 operations)
    2, 1, 3;   % Job 2: M2 -> M1 -> M3       (3 operations)
    1, 3, 0;   % Job 3: M1 -> M3             (2 operations)
];

% Processing time matrix (duration of each operation)
M_tiempos = [
    10, 8, 4;  % Job 1: 10u on M1, 8u on M2, 4u on M3
     5, 5, 5;  % Job 2:  5u on M2, 5u on M1, 5u on M3
     7, 9, 0;  % Job 3:  7u on M1, 9u on M3
];
```

1.2. Constraints

A feasible schedule must satisfy:

1. **Precedence constraints:** The operations of each job must be executed in the order prescribed by $\mathbf{R}$. That is, for job j, operation $k+1$ cannot start before operation k has finished.
2. **Machine capacity:** Each machine can process at most one operation at a time. No two operations assigned to the same machine may overlap in time.
3. **Non-preemption:** Once an operation starts on a machine, it runs to completion without interruption.

1.3. Objective Function

Let C_j be the completion time of the last operation of job j. The objective is to minimize the makespan:

$$\textbf{Minimize } C_{\text{máx}} = \max_{j=1,\ldots,J} C_j \tag{3}$$

1.4. Representation: Operation-Based Encoding

The chromosome is an integer vector of length N_{op}, where each gene is a job index $j \in \{1, \ldots, J\}$. Each job j appears exactly n_j times in the chromosome. Reading the chromosome from left to right, the q-th occurrence of job j refers to the q-th operation of that job.

Example: Decoding a chromosome

Consider the instance above ($J = 3$, job 1 and 2 have 3 operations, job 3 has 2).
Chromosome: $\vec{x} = [1,\ 3,\ 2,\ 1,\ 2,\ 3,\ 1,\ 2]$

Decoding (left to right):

1. Gene = **1** (1st occurrence of job 1) → Schedule op. 1 of job 1 on M1 for 10u.
2. Gene = **3** (1st occurrence of job 3) → Schedule op. 1 of job 3 on M1 for 7u.
3. Gene = **2** (1st occurrence of job 2) → Schedule op. 1 of job 2 on M2 for 5u.
4. Gene = **1** (2nd occurrence of job 1) → Schedule op. 2 of job 1 on M2 for 8u.
5. Gene = **2** (2nd occurrence of job 2) → Schedule op. 2 of job 2 on M1 for 5u.
6. Gene = **3** (2nd occurrence of job 3) → Schedule op. 2 of job 3 on M3 for 9u.
7. Gene = **1** (3rd occurrence of job 1) → Schedule op. 3 of job 1 on M3 for 4u.
8. Gene = **2** (3rd occurrence of job 2) → Schedule op. 3 of job 2 on M3 for 5u.

Each operation is scheduled at the earliest feasible time, i.e. the maximum of (a) the completion of the preceding operation of the same job, and (b) the earliest availability of the required machine.

1.5. Decoding: Greedy Left-to-Right Scheduling

Given a chromosome $\vec{x}$, the schedule is built by the following deterministic procedure:

1. Initialise two vectors: $\texttt{job_time}(j) = 0$ for $j = 1, \ldots, J$ (tracks when each job's last operation finished) and $\texttt{machine_time}(m) = 0$ for $m = 1, \ldots, M$ (tracks when each machine becomes free).
2. Initialise a counter vector: $\texttt{op_count}(j) = 0$ for $j = 1, \ldots, J$.
3. For each gene x_i in the chromosome (from $i = 1$ to N_{op}):
   a) Let $j = x_i$. Increment $\texttt{op_count}(j)$, so $k = \texttt{op_count}(j)$.
   b) Look up machine $m = R_{j,k}$ and duration $d = T_{j,k}$.
   c) Compute start time: $s = \text{máx}(\texttt{job_time}(j),\ \texttt{machine_time}(m))$.
   d) Update: $\texttt{job_time}(j) = s + d$ and $\texttt{machine_time}(m) = s + d$.
4. Return $C_{\text{máx}} = \text{máx}_j\ \texttt{job_time}(j)$.

2. Function Signature and Deliverable

Your submission must consist of a single MATLAB file named `solve_int_ga.m` implementing the following interface exactly:

Listing 2: Required function signature for `solve_int_ga.m`

```
function [best_sol, best_cost, log, num_gens] = ...
    solve_int_ga(M_maquinas, M_tiempos, time_limit)
```

2.1. Inputs

Cuadro 1: Input parameters to `solve_int_ga`

Parameter	Size	Description
`M_maquinas`	$J \times M$	Machine routing matrix. Entry (j, k) is the machine for the k-th operation of job j. Value 0 means no operation.
`M_tiempos`	$J \times M$	Processing time matrix. Entry (j, k) is the duration of the k-th operation of job j. Value 0 if no operation.
`time_limit`	1×1	Time budget [s].

2.2. Outputs

Cuadro 2: Output values returned by `solve_int_ga`

Output	Size	Description
`best_sol`	$1 \times N_{op}$	Integer permutation chromosome of the best solution found (job indices).
`best_cost`	1×1	Makespan ($C_{\text{máx}}$) of `best_sol`.
`log`	$1 \times K$	Best cost found at each generation (convergence trace).
`num_gens`	1×1	Total number of generations completed.

Skeleton

Listing 3: Recommended skeleton for `solve_int_ga.m`

```
function [best_sol, best_cost, log, num_gens] = ...
    solve_int_ga(M_maquinas, M_tiempos, time_limit)

    [J, M] = size(M_maquinas);

    % --- 1. Compute number of operations per job ---
    n_ops = sum(M_maquinas > 0, 2)';    % 1xJ vector
    N_op  = sum(n_ops);                  % total operations

    % --- 2. Build base chromosome (unshuffled) ---
    base = [];
    for j = 1:J
        base = [base, repmat(j, 1, n_ops(j))];
    end

    % --- 3. Initialise population ---
    pop_size = 50;
    pop = zeros(pop_size, N_op);
    for i = 1:pop_size
        pop(i,:) = base(randperm(N_op));
    end

    best_cost = Inf;
    best_sol  = base;
    log       = [];
    num_gens  = 0;
    t_start   = tic;

    while toc(t_start) < time_limit - 0.5
        num_gens = num_gens + 1;

        % --- 4. Evaluate population (decode + compute makespan) ---

        % --- 5. Selection ---

        % --- 6. Crossover (e.g. OX, PMX) ---

        % --- 7. Mutation (e.g. swap, insert) ---

        % --- 8. Elitism ---

        % --- 9. Update best ---
        log(end+1) = best_cost;
    end
end
```